Koneru Lakshmaiah Education Foundation
(Deemed to be University estd. u/s. 3 of the UGC Act, 1956)
Off-Campus: Bachupally-Gandimaisamma Road, Bowrampet, Hyderabad, Telangana - 500 043.
Phone No: 7815926816, www.klh.edu.in

OPERATING SYSTEMS AND SYSTEMS PROGRAMMING (25CS2104E)

2026–27, Term-I

Project Problem Statement Submission Form

Section / Team Details

Section No.: 2

Team No.: 5

Project Title: Linux File Descriptor and File I/O Management System

1. **Team Members**

Roll Number	Student Name	Signature
2520030587	J.Sai Spoorthi	
2520030514	N.Sai Pranavi	
2520030591	T.Sudeshna	

2. **Abstract**

The **Linux File Descriptor and File I/O Management System** is a Linux-based system programming project that demonstrates how the operating system manages files through **File Descriptors (FDs)** and **File I/O operations**. In Linux, when a process opens a file, the kernel assigns a file descriptor, which the process uses to access that opened file. Understanding this process is important for learning Operating System concepts such as **system calls, process-level file management, kernel interaction, and input/output operations**. The proposed system provides a simple way to open, read, write, and close files using Linux system calls such as open(), read(), write(), and close(). It displays the file name, assigned file descriptor, and the operations performed on the file, making the internal file management process easier to understand. The system can also manage multiple opened files by maintaining their corresponding file descriptors and basic file information. The expected outcome is a practical Linux application that demonstrates how user programs interact with the Linux kernel for file management and how file descriptors are used during File I/O operations. This project provides hands-on understanding of **Linux system programming and file management** through a simple and demonstrable implementation.

2. Problem Statement

In Linux, applications cannot directly access files without interacting with the operating system. When a process opens a file, the Linux kernel assigns a **File Descriptor (FD)**, which is then used to perform operations such as reading, writing, and closing the file. However, understanding how file descriptors are assigned and how File I/O operations are managed can be difficult for beginners, especially when multiple files are opened at the same time.

The problem addressed by this project is the **lack of a simple and practical system to demonstrate and manage the relationship between files, file descriptors, and File I/O operations**. The project aims to provide a Linux-based solution that allows users to observe how files are opened, how FDs are assigned, how data is read or written using those FDs, and how FDs are released when files are closed.

This project is relevant to **Operating Systems and Systems Programming** because it demonstrates important concepts such as **system calls, kernel-level file management, file descriptors, process resources, and File I/O** in a practical Linux environment.

3. Objectives

1. **To understand and demonstrate** how Linux assigns and manages File Descriptors when files are opened.
2. **To implement basic File I/O operations** such as opening, reading, writing, and closing files using Linux system calls.
3. **To monitor and display file information**, including file names, assigned File Descriptors, and the operations performed on them.
4. **To provide practical understanding of Operating System concepts** such as system calls, process-level file management, File Descriptors, and Linux File I/O.

4. Proposed Methodology

1. The project will be developed in **Ubuntu/Linux using C programming and Linux system calls**. The system will provide a simple menu-driven interface for performing file operations. When the user selects a file, the program will use the open() system call to open it, and Linux will assign a **File Descriptor (FD)**. The system will display the file name and assigned FD and use it for further operations. The user can then perform **read, write, and close** operations using read(), write(), and close() system calls. The system will also handle errors such as invalid files or unsuccessful operations. After closing a file, the corresponding FD will be released.

1. Start
2. Select/Create File
3. Open File using open()
4. Linux assigns File Descriptor
5. Display File Name + FD
6. Read / Write File
7. Close File using close()
8. FD Released
9. End

5. Operating Systems Concepts / Linux APIs Used

OS Concept / Linux API / System Call	Purpose in the Project
File Descriptor(FD)	Identifies an opened file and is used to perform File I/O operations.
Open()	Opens a file and returns a File Descriptor.
Read()	Reads data from an opened file using its FD.
Write()	Writes data to an opened file using its FD.
Close()	Closes the opened file and releases its FD.
System Calls	Provides communication between the user program and Linux kernel for file operations.
Linux Kernel	Manages files, File Descriptors, and I/O requests made by the program.
File I/O	Handles reading and writing data between the program and files.
Error Handling (errno)	Helps identify and handle errors during file operations.
Process File Descriptor Table	Maintains the relationship between File Descriptors and opened files for a process.

POSIX APIs	Provides standard interfaces for performing file and system-level operations in Linux.

6. Individual Contribution

Roll Number	Student Name	Individual Responsibility
2520030587	J.Sai Spoorthi	**File Descriptor & File Opening Module** – Implement open(), File Descriptor assignment, and display file information.
2520030514	N.Sai Pranavi	**File I/O Module** – Implement read() and write() operations and handle file data and errors.
2520030591	T.Sudeshna	**FD Management & Integration Module** – Implement close(), FD tracking, menu-driven interface, integration, and testing.

7. Tools / Platforms / Software Used

Tool / Platform / Software	Purpose
Linux / Ubuntu	Operating system environment for developing and running the project.
C Programming	Used to implement File Descriptor and File I/O operations.
GCC Compiler	Compiles the C source code into an executable program
Linux System Calls	Used for file operations such as open(), read(), write(), and close().
POSIX APLs	Provides standard interfaces for Linux file and system operations.
Terminal	Used to compile, execute, and test the project.
Nano / Vim	Used to create and edit C source files.
Make / Makefile	Can be used to automate compilation of the project.
Git / GitHub	Used for version control and maintaining project source code.

8. Expected Outcome

*The proposed system is expected to successfully demonstrate **Linux File Descriptor and File I/O management** in a simple and practical way. The system will allow users to open, read, write, and close files using Linux system calls.*

The expected outputs are:

1. *Display the **file name and File Descriptor (FD)** assigned by Linux.*
2. *Successfully perform **read and write operations** using the assigned FD.*
3. *Display appropriate messages for successful and unsuccessful file operations.*
4. *Release the File Descriptor when the file is closed.*
5. *Provide a clear understanding of how **Linux kernel, File Descriptors, system calls, and File I/O** work together.*

Open File

Linux assigns FD

Display FD

Read / Write

Close File

FD Released

9. GitHub Repository

GitHub Repository Link: https://github.com/pranavinaliveni05-dot/514_587_591_OSSP

Repository Created:

☐ Yes

Faculty Name and Signature

Faculty Name: Dr.K.Anusha

Signature: _____

Remarks: _____

Faculty Use

Project Approved: _____

☐ Yes ☐ No

Date: _____

Faculty Signature: _____